

Extension Methods

NET 022 - Advanced C# Programming

Prepared By: Supun Kandaudahewa
supun.kandaudahewa@humber.ca



WE ARE

HUMBER

Agenda

- Extension Methods – The Use Case
- Writing Extension Methods and Tips
- Using Extension Methods
- Extending
 - Interfaces
 - Generics
- Chaining Extension Methods
- Limitations of Extension Methods
- Coding Exercise

Why we need Extension Methods

- When developing applications, we might come across scenarios where we want to add functionality to the existing behavior.
- In the below illustration, assume that ClassA (containing 3 methods) is already tested and deployed.
- Now we want to add more functionality to it. We need two more methods added.
- Let's talk about our options here.



Option 1 – Edit Class A

- It might be easier to change ClassA and include the two new methods. But it can lead to many drawbacks.
- Cons of modifying ClassA
 - ClassA needs to be recompiled and any other class which references ClassA needs to be recompiled and redeployed.
 - You might not have access to the source code.
 - You might not have permissions to edit the code.
 - Testing has been already done and need to redo testing from scratch.
 - Violates Open-Closed principle

Solution Using Inheritance → take ClassA as base class and derive ClassB from it. Then add new methods to ClassB.

Option 2 – Using Inheritance

- Using inheritance might be a better option. But it also has many negative effects.
- Cons of using inheritance to solve the problem.
 - Sealed classes cannot be inherited. String etc.
 - Structs cannot be inherited.
 - Base Instance can only be used to call 3 methods (A,B, C). It cannot be used call D and E.

Base

ClassA
MethodA
MethodB
MethodC

Derived

ClassB:ClassA
MethodA, MethodB
MethodC
MethodD and MethodE

Option 3 – Using Extension Methods

- You don't need the source code.
- You don't need any permissions as you are not modifying any existing code.
- Original type does not require recompilation.
- Can be used to extend the functionality of:
 - Classes and Structs
 - Interfaces
 - Generics → Classes and Interfaces

Writing Extension Methods

- The class containing the extension and the extension method must be a **static**.
- Type to extend is specified by the “**this**” keyword.
- Extension methods can live in any namespace or assembly.
- Extension methods must be defined in a top-level static class and not in nested classes.
- Usually, extension classes are named adhering to following syntax **[ExtendedType].[Extensions] → MyLibraryExtensions**
- Extension methods can be overloaded as long as the parameters are different.
- If you declare a method with the same name in base class and in the extended class, base class method will take precedence.
- Though extension methods are static, once they are bound with a class through a bounding parameter, they become non-static.

7

<https://dotnetfiddle.net/4gmVEQ>

Extending Interfaces

- Using extensions on interfaces means **any class implementing that interface will also inherit the extension.**
- Specify the interface to extend in the **`this`** argument.
 - Create an interface `IVehicle` having a property **Name** and a method which returns void named `Drive()`
 - Create 2 classes `Car` and `Truck` inheriting from `IVehicle`
 - `Car` constructor will take two strings `make,model` and set the `Name` property.
 - `Truck` constructor will take two strings `make,model,trim` and set the `Name` property.
 - `Drive` method should have `→ Console.WriteLine($"{Name} is driving.");`
 - Let's extend the `IVehicle` to add a `Reverse` method `→ Console.WriteLine($"{Name} is reversing.");`

Extending Generics

- Let's extend `List<T>` to demonstrate this.
- Generic classes can be extended the same way as non-generic classes and interfaces.

<https://dotnetfiddle.net/w12FCs>

Chaining Extension Methods

- Extension methods, provide a tidy way to chain functions.
- These functions are evaluated from Left to right.
- Type returned by left hand side method should be compatible with the next method.

<https://dotnetfiddle.net/oevvbA>

Limitations

- You cannot bypass the accessibility.
- You can't import one extension from a namespace without getting all the rest.
- It's not immediately obvious from the source code where the method is defined.
- If you're extending a 3rd party class and in a later version they introduce a new method with the same signature, you won't easily know that the meaning of your calling code has changed.
 - As said in the previous slide previously written method will take precedence over the static method with same name.

THANK YOU.



WE ARE

HUMBER